ICPSwap V3 Service — Security Audit Report v7.0

Audit Date: April 26, 2026 **Branch:** `feature/one-step-swap` **Auditor:** Independent security review (post-v6.2) **Scope:** Full review of all `src/` Motoko sources (~7,885 lines) **Methodology:** v6.2 fix verification + adversarial review of all public/private entry points + state-machine correctness review + cross-canister flow analysis

---

Executive Summary

This audit was conducted as a follow-up to the v6.2 audit dated 2026-04-23. It performs two tasks:

1. **Verification** of all 29 v6.2 "FIXED" remediations, plus reassessment of the 5 "ACCEPTED" items.
2. **Adversarial review** under the highest-tier audit standard (full state-machine, atomicity, MEV, upgrade-interruption, and ambiguous-catch coverage) to surface any remaining issues.

Headline Numbers

Severity	Count	Prior Status
Critical	0	—
High	4	1 incomplete v6.2 fix + 3 newly identified
Medium	8	4 newly identified + 4 v6.2 boundary cases
Low	9	newly identified
Informational	3	newly identified
Total new findings	**24**	
v6.2 FIXED items reverified	29	28 PASS, 1 PARTIAL (NEW-10)
v6.2 ACCEPTED items reaffirmed	5	5 PASS (no change in risk posture)

Top-line Conclusions

- **NEW-10 from v6.2 is incompletely fixed.** The new `await _refund(...)` in `deleteFailedTransaction` does not actually catch the failure mode the v6.2 audit described, because the underlying `_refund` returns `#ok` as soon as the refund is *scheduled*, not when the token transfer actually succeeds. This is the most important finding of v7.0 and is graded **High**.
- **Three new High-severity issues** were identified outside the v6.2 scope:
  - `SwapFeeReceiver._commonSwap` and `_ICRC1SwapToICP` perform protocol-fee swaps with `amountOutMinimum = "0"` — value leak through MEV-style price manipulation.
  - `SwapDataBackup.backup` performs a multi-await snapshot with no pool quiescence — recovery from such a backup would yield inconsistent state.
  - `SwapPoolInstaller.install` is missing the explicit `Cycles.add` calls that the local-install path in `SwapFactory` uses; depending on `IC0Utils` semantics this may either silently create cycle-less canisters or be redundant. **Requires confirmation against `IC0Utils` source.**
- **DeletedSwapPool** has two new issues introduced by the v6.2 NEW-25 fix: `retryFailedRefunds` lacks the `_isRefunding` mutex; and `postupgrade` resumes refund processing without re-fetching the (non-stable) fee cache.
- The v6.2 NEW-02 fix introduced a generation counter that is correctly decremented in two of three pending-clear paths, but skipped on the max-retry exit path.

Funds-at-Risk Summary

The protocol's user-funds-at-risk surface is **smaller than it appears in the finding count**:

- `NEW7-01` does not directly cause user funds to be lost, but it **causes the admin to be misinformed about the state of refund flows** and creates conditions under which an admin double-refund could be issued. This is a control-plane integrity issue, not a direct theft vector.
- `NEW7-07` causes ongoing slow leakage of *protocol-owned* fees to MEV actors. User funds are not directly at risk.
- `NEW7-08` is a recovery-path issue: in the absence of pool corruption, it does not cause loss; in the presence of pool corruption, it may cause loss during recovery.
- `NEW7-09` would, if confirmed, leak admin cycles, not user funds.

No critical user-fund theft vector was discovered in this round.

Methodology

What was audited

- **Files audited fully or in substantial part (~5,400 LOC):**
  - `SwapPool.mo` (3,323 LOC) — full read of all public entry points, all state mutators, init/upgrade hooks, permission helpers, math libraries' call sites, and the entire swap/limit-order flow.
  - `DeletedSwapPool.mo` (258 LOC) — full read.
  - `SwapFactory.mo` (1,171 LOC) — full read of pool creation, upgrade orchestration, installer management, wasm management, permission model.
  - `SwapFeeReceiver.mo` (974 LOC) — full read of claim/swap/burn flows, permission model, lock semantics.
  - `PasscodeManager.mo` (354 LOC) — full read.
  - `PositionIndex.mo` (296 LOC) — full read.
  - `SwapDataBackup.mo` (319 LOC) — full read.
  - `SwapPoolInstaller.mo` (137 LOC) — full read.
  - `TrustedCanisterManager.mo` (64 LOC) — full read.
  - `components/transaction/lib.mo` (852 LOC) — full read of state machines.
  - `components/transaction/{Deposit,OneStepSwap,Refund,...}.mo` — read of state-transition logic.
  - `components/{ICRC21,Job,TokenHolder,WasmManager,PositionTick}.mo` — full or substantial read.
  - `utils/{AccountUtils,PoolUtils}.mo` — full read.
  - `libraries/FullMath.mo` — full read of mulDiv.
- **Files NOT independently audited:**
  - `Types.mo` (treated as data definitions only).
  - `libraries/{TickMath, SwapMath, SqrtPriceMath, LiquidityMath, LiquidityAmounts, Tick, TickBitmap, BlockTimestamp, FixedPoint128}.mo` —

these are well-trodden Uniswap V3 math primitives and were assumed correct for this round; v6.2 already covered them. Risk is residual and limited to numerical edge cases.

- `components/{TokenAmount, SwapRecord, UpgradeTask, PoolData}.mo` — supporting components; spot-checked only.
- `SwapFactoryValidator.mo` — validator class, not part of runtime execution.

What this report does *not* cover

- Front-end / SDK integration risks.
- Cycle economics / DoS via cycle drain (other than the specific finding called out).
- Long-running adversarial price-manipulation simulation requiring on-chain state.
- Subnet-level concerns (replica trust, NNS).

---

v6.2 FIX VERIFICATION TABLE

The v6.2 audit reported 29 FIXED + 5 ACCEPTED items (34 findings total). I reverified each. **Bold** indicates items where this audit's finding diverges from v6.2's claim.

ID	v6.2 Status	v7.0 Verification	Notes
NEW-01	FIXED	✅ **PASS**	`try/catch` wrappers correctly placed around `await _swap` in both `depositFromAndSwap` (line 1773–1794) and `depositAndSwap` (line 1855–1875). Catch block correctly calls `oneStepSwapSwapFailed` then `_refund`.
NEW-02	FIXED	⚠️ **PARTIAL PASS**	Generation mechanism present. However, the max-retry exit path at SwapPool.mo:264 clears `_pendingExecution` without incrementing `_pendingGeneration`, contrary to the fix description. See **NEW7-04**.
NEW-03	FIXED	✅ PASS	`_isTxFailed` helper correctly added (line 3188); `_clearExpiredFailedTransactionJob` correctly preserves non-failed expired txs with WARN log.
			Outer guard `if (info.status != #Completed and`

NEW-04	FIXED	✅ PASS	`info.status != #Failed)` correctly added to `#OneStepSwap` branch in `setFailed` (lib.mo:838).
NEW-05	FIXED	✅ PASS	`deposit_and_swap_consent_msg` now takes `request.method` and parameterizes the displayed name (ICRC21.mo:48–49, 116).
NEW-06	FIXED	✅ PASS	`assert(stagingWasmBlob.size() > 0)` correctly added at WasmManager.mo:66.
NEW-07	ACCEPTED	✅ Re-affirmed	Risk posture unchanged. The within-session uploader isolation is sufficient given the narrow attack window (admin-only, upgrade-during-upload).
NEW-08	FIXED	✅ PASS	`deletedEntry` correctly saved and restored in `removeLimitOrder` (SwapPool.mo:2021–2086). Three-tuple form `?(LimitOrderType, key, value)` correctly handles both upper and lower lists.
NEW-09	ACCEPTED	✅ Re-affirmed	Governance `transfer` function provides reconciliation path. Admin operationally must monitor logs. **One newly noted gap:** `depositFrom` lacks the `_addLog` call that V6 NEW-19 added to `deposit`. See **NEW7-10**.
NEW-10	FIXED	❌ **FAIL**	The `await _refund(...)` returns `#ok` from `_refund` as soon as `_tokenHolderService.withdraw` succeeds and the timer is scheduled (SwapPool.mo:1093). It returns *before* the actual `tokenAct.transfer` is awaited. Admin gets `#ok(true)` when refund is initiated, not when refund actually completes. See **NEW7-01**.
NEW-11	FIXED	✅ PASS	All four refund call sites in `depositFromAndSwap` / `depositAndSwap` now use `depositAmount` (lines 1768, 1780, 1786, 1792, 1849, 1861, 1867, 1873).
NEW-12	ACCEPTED	✅ Re-affirmed	`updatePoolIds` is read-only sync. **Adjacent issue**: separate functions `addPoolId`, `removePoolIdWithoutCheck` are also unauthenticated and have larger surface — see

			NEW7-11 and **NEW7-12**.
NEW-13	FIXED	✅ PASS	Three `_log` calls correctly added at SwapPool.mo:1021, 1024, 1027 for the OneStepSwap withdraw error branches.
NEW-14	ACCEPTED	✅ Re-affirmed	All call sites guarantee non-zero denominator. Acceptable for the current call graph; brittle to future modification.
NEW-15	FIXED	✅ PASS	Comment correctly placed at the `_isAvailable` controller bypass.
NEW-16	FIXED	✅ PASS	`[transfer]` operation tag, recipient principal, "ambiguous" label all present in PasscodeManager.mo:289–293.
NEW-17	FIXED	✅ PASS	`_parseVersion` helper added (SwapFactory.mo:424–433). Note: helper traps on malformed input rather than returning `Result`. See **NEW7-06** for stylistic concern.
NEW-18	FIXED	✅ PASS	`_upgradeWasmSnapshot` correctly snapshotted at `setUpgradePoolList` line 516 and used at `_execUpgrade` line 884.
NEW-19	FIXED	⚠️ **PARTIAL PASS**	`_addLog` correctly added to `deposit` catch (PasscodeManager.mo:142). However, `depositFrom` catch block at PasscodeManager.mo:164–167 still lacks the equivalent `_addLog` call. See **NEW7-10**.
NEW-20	FIXED	✅ PASS	`Array.find` replaced with `HashMap` lookup in `SwapRecord.deleteSyncedData`.
NEW-21	FIXED	✅ PASS	Documentation comment present.
NEW-22	FIXED	✅ PASS	Comment present documenting trap intentionality.
NEW-23	FIXED	✅ PASS	Dead function deleted. `Result` import removed.
NEW-24	FIXED	✅ PASS	Loop condition correctly tightened to `ind + 1 < 32` in AccountUtils.mo:13.

NEW-25	FIXED	✅ **PASS (main logic)**	`_failedRefunds` tracking, `transferAll` blocking, `retryFailedRefunds`, `getFailedRefunds` all correctly added. **However**, two new boundary issues introduced — see **NEW7-02** and **NEW7-03**.
NEW-26	FIXED	✅ PASS	`_isPublicQuery` helper present and correctly classifies read-only methods (SwapPool.mo:3287–3312).
NEW-27	ACCEPTED	✅ Re-affirmed	Single hardcoded controller is the design. IC platform controller provides ultimate recovery.
NEW-28	FIXED	✅ PASS	`resetSyncingFlag()` at SwapFeeReceiver.mo:894–897. `postupgrade` resets `_isSyncing := false` at line 942.
NEW-29	FIXED	✅ PASS	`forceReleaseLock()` at SwapFeeReceiver.mo:899–902. `postupgrade` resets `_locked := false` at line 941.
NEW-30	FIXED	✅ PASS	Both interval setters reject values below 3600 seconds (SwapFeeReceiver.mo:870, 877).
NEW-31	FIXED	✅ PASS	Balance-zero check added (SwapFeeReceiver.mo:352). `fee = null` retained as design choice.
NEW-32	FIXED	✅ PASS	`_hasPermission(caller)` check added to `getPoolBackup` and `getAllPoolBackups`.
NEW-33	FIXED	✅ PASS	`Principal.isAnonymous` check added in `addCanister` (TrustedCanisterManager.mo:18).
NEW-34	FIXED	✅ PASS	`assert(activeWasm.size() > 0)` correctly placed before `create_canister` in both Local install and `SwapPoolInstaller.install`.

Verification summary: 28/29 PASS, 1/29 FAIL (NEW-10 incompletely fixed → escalated to NEW7-01 High).

NEW FINDINGS (v7.0)

High (4)

NEW7-01 — `await _refund` in `deleteFailedTransaction` cannot detect actual transfer failure (NEW-10 incomplete fix)

- **Severity:** High
- **Location:** `SwapPool.mo:2438-2497` (`deleteFailedTransaction`), `SwapPool.mo:1042-1107` (`_refund`)
- **Description:**

  The v6.2 NEW-10 fix changed two `ignore _refund(...)` calls in the `#OneStepSwap` branch of `deleteFailedTransaction` to `await _refund(...)`, with the stated intent: *"On failure: logs error and returns #err to admin."*

  Inspecting `_refund` reveals that the function does not actually `await` the `tokenAct.transfer` call inside its own scope. The actual transfer is fired from inside a closure scheduled via `Timer.setTimer(...)`:

```
// _refund body (SwapPool.mo:1042-1107)
func __refund<s>() {
    ignore Timer.setTimer<s>(#nanoseconds(0), func (): async () {
        try {
            ...
            switch (await tokenAct.transfer({...})) {
                case (#Ok(index)) { /* update tx state */ };
                case (#Err(e)) { ignore _txState.refundFailed(txIndex, ...); }
            };
        } catch (e) { ignore _txState.refundFailed(...); };
    });
};

if (amount <= fee) { return #err(#InternalError("Amount less than fee")); };
let txIndex = _txState.startRefund(...);

if (_tokenHolderService.withdraw(to.owner, token, amount)) {
    _txState.refundCredited(txIndex);
    ...
    __refund<s>();              // schedules timer, does NOT await transfer
    return #ok(amount);         // <-- returns BEFORE transfer is attempted
    ...
} else {
    ignore _txState.refundFailed(txIndex, "Insufficient balance");
    return #err(#InternalError("Insufficient balance"));
```

```
};
```

Therefore `_refund` returns:

- `#err` only if `amount <= fee` or the internal `_tokenHolderService.withdraw` debit fails
- `#ok(amount)` as soon as the timer is scheduled, regardless of whether the actual ICRC1 transfer ever succeeds

When `deleteFailedTransaction(txId, refund=true)` calls `await _refund(...)` and receives `#ok`, the admin caller receives `#ok(true)` indicating successful cleanup. But the actual transfer may later fail in the timer (token canister BadFee, Duplicate, TemporarilyUnavailable, network error, etc.), and the only feedback is a `_txState.refundFailed(...)` call that the admin must proactively poll.

- **Impact:**
  1. **Admin-control-plane integrity is compromised.** The admin running the `deleteFailedTransaction` workflow has no way to distinguish between "refund actually completed" and "refund initiated but later silently failed." A typical operational pattern (call `deleteFailedTransaction`, mark the case as resolved on `#ok`) leaves failed refunds silently stuck in `#Refund(status=#Failed)` state.
  2. **Risk of double-refund / pool drain on operational error.** If the admin sees `#ok` and assumes resolution, but the user complains and the admin re-runs cleanup, the second call will either:
     - Find the refund in a non-`Failed` state and refuse to retry, or
     - If the admin manipulates state (e.g., resets the failed refund), credit the user a second time — the user has been credited internally twice but the `_tokenHolderService.withdraw` succeeds twice (debiting only once on each path), pulling more from the pool's main account than the user is entitled to.

     The exact outcome depends on the admin recovery procedure; the danger is that the admin is operating on bad information.
  3. **For the `#Deposit`, `#Withdraw`, `#Refund` branches at lines 2450–2463, the calls remain `ignore _refund(...)` — these branches were not part of the v6.2 NEW-10 fix at all.** Those are even more silent: admin gets `#ok(true)` from the outer function with no error propagation whatsoever.
- **Recommendation (one of):**

1. **(Best) Synchronize `_refund`** — restructure `_refund` so the actual `tokenAct.transfer` is awaited within the function body, not in a separate timer. Return `#ok` only after on-chain transfer succeeds. This requires careful ordering of the internal `_tokenHolderService.withdraw` and the external transfer (use the standard "debit then transfer; restore on failure" pattern that already exists in `PasscodeManager.withdraw`).
2. **(Defensive)** Keep the timer-based pattern but change the function name and return semantics: rename `_refund` to `_initiateRefund`, return `#ok({txIndex; status = #Pending})`, and require admin to poll. Update `deleteFailedTransaction` to return a structured response that conveys "initiated, not yet completed."
3. **(Operational)** Add a periodic job (or admin-callable function) that scans `_txState` for `#Refund(status=#Failed)` entries and either auto-retries with a backoff or alerts via a structured query.

- **Status:** New finding (v6.2 fix incomplete).

NEW7-07 — Protocol-fee swaps in `SwapFeeReceiver` use `amountOutMinimum = "0"`, leaking fees to MEV

- **Severity:** High
- **Location:** `SwapFeeReceiver.mo:411`, `SwapFeeReceiver.mo:492`
- **Description:**

  Both `_ICRC1SwapToICP` and `_commonSwap` perform the protocol's own swap legs (collected fees → ICP → ICS) with no slippage protection:

  ```
  // _commonSwap (SwapFeeReceiver.mo:489-492)
  switch (await poolAct.swap({
      zeroForOne = ...;
      amountIn = debug_show(depositedAmount);
      amountOutMinimum = "0";        // <-- no slippage protection
  })) { ... };
  ```

  The corresponding pool's `swap` function honors `amountOutMinimum` only when it's > 0 (SwapPool.mo:2285–2291). Thus the protocol's claim+swap pipeline accepts arbitrary slippage on every claim cycle.

- **Attack scenario (IC-specific):**

IC does not have a public mempool, so traditional Ethereum sandwiching is harder. However:

1. The interval-based trigger of `_autoSyncPools` (24-hour recurring timer) and `_autoClaim` is fully predictable from public state (`_lastSyncTime` , `_icpPoolClaimInterval`).
2. A patient adversary can co-locate a watcher canister, observe the cycle approaching, and execute a price-pushing swap immediately before the protocol's swap, then unwind immediately after.
3. Inter-canister call ordering on the same subnet provides reasonable confidence that the attack swap → protocol swap → unwind sequence executes in order if the attacker controls a high-priority canister.
4. The amount of value extractable scales with how much fee is held in the receiver canister. For a popular pool, this can be a significant fraction of weekly trading fees.

- **Impact:**

  Slow, ongoing leakage of protocol fees. Exact magnitude depends on token pair liquidity, claim interval, and adversarial sophistication. Estimated 0.5–5% of every claim cycle on low-liquidity pairs; sub-1% on high-liquidity pairs.

  This is **protocol-owned funds**, not user funds.

- **Recommendation:**

  1. Quote price via `_factoryAct.getPool(...)` + read-only `quote` on the pool, compute a reasonable lower bound (e.g., 99% of quoted out for high liquidity, 95% for low), and pass that as `amountOutMinimum` .
  2. If quote is unavailable (pool error), abort the swap rather than executing with zero protection.
  3. Optionally add a periodic-jitter to the trigger time to reduce predictability.
  4. Maintain a per-pool max-slippage policy (admin-configurable).

- **Status:** New finding.

NEW7-08 — `SwapDataBackup.backup` is non-atomic; recovery from such backup yields inconsistent state

- **Severity:** High (conditional on use of recovery path)

- **Location:** `SwapDataBackup.mo:59-175`
- **Description:**

  `backup` performs a sequence of approximately 14 separate `await poolAct.xxx()` calls to assemble the snapshot:

  ```
  let metadata          = await poolAct.metadata();           // T0
  let allTokenBalances  = await poolAct.allTokenBalance(...); // T1
  let positions         = await poolAct.getPositions(...);    // T2
  ...
  let withdrawQueue     = await poolAct.getWithdrawQueueInfo(); // T13
  ```

  Each `await` is a separate inter-canister round-trip and a separate Motoko message boundary. Between any two consecutive awaits, the pool may be processing user deposits, swaps, claims, withdraws, etc. **No quiescence is enforced** (the pool is not made `_available = false` for the duration).

  The resulting `PoolBackupData` therefore mixes data from up to ~14 distinct moments in time. For example, the snapshot's `allTokenBalances` may reflect state before a deposit, while `positions` / `userPositions` reflect state after that deposit.

- **Impact:**

  The recovery path (`recoverPool`) is currently commented out (lines 217–248), so this finding is **not currently exploitable**. However:

  1. The entire purpose of `SwapDataBackup` is to enable disaster recovery. If `recoverPool` is ever activated to restore a corrupted pool, the inconsistency could:
     - Mint or destroy liquidity that doesn't match token reserves
     - Skip or duplicate user balances
     - Restore positions that reference user states the backup didn't capture
  2. Even without recovery, the backup is consumed by `_execUpgrade` (`backupCid.isBackupDone(poolCid)` is checked in the upgrade pipeline). A successful `isBackupDone` does not guarantee a *useful* backup.
  3. `governanceCid` and admin operators reading the backup for offline analysis may make incorrect decisions based on inconsistent data.

- **Recommendation:**

  1. **Make pool quiescent during backup.** Add a `setAvailable(false)` at the start of

`backup` and `setAvailable(true)` at the end (with try/catch to ensure restoration). The `SwapFactory._execUpgrade` already has a `turnOffAvailable` step — `backup` should use the same primitive *before* querying.

2. **Or, add a snapshot-id mechanism** in the pool: a query that returns a consistent state along with a sequence number; the backup polls until two consecutive snapshots agree.
3. **Document that the current `backup` is best-effort** and not guaranteed consistent if the live pool is active.

- **Status:** New finding.

NEW7-09 — `SwapPoolInstaller.install` may be missing `Cycles.add` before `create_canister` / `deposit_cycles` (REQUIRES VERIFICATION)

- **Severity:** High *if confirmed* — Informational *if `IC0Utils` handles it internally*
- **Location:** `SwapPoolInstaller.mo:50-53`
- **Description:**

  Compare two pool-creation paths:

  Local install (SwapFactory.mo:1004–1008):

```
Cycles.add<s>(_initCycles);                                          // (a
let createCanisterResult = await IC0Utils.create_canister(null, null, _initCyc
let canisterId = createCanisterResult.canister_id;
await IC0Utils.deposit_cycles(canisterId, _initCycles);              // (b
```

  External installer (SwapPoolInstaller.mo:50–53):

```
let createCanisterResult = await IC0Utils.create_canister(null, null, _initCyc
let canisterId = createCanisterResult.canister_id;
await IC0Utils.deposit_cycles(canisterId, _initTopUpCycles);
```

  Standard Motoko convention is that `ExperimentalCycles.add(amount)` is required *before* an inter-canister call to attach cycles to that call. The factory's local path follows this pattern; the installer does not.

  Either:

  - **(A)** `IC0Utils.create_canister(_, _, n)` and `IC0Utils.deposit_cycles(_, n)`

internally call `ExperimentalCycles.add` based on their last argument. Then the installer is correct and the factory's `Cycles.add` is redundant. The `IC0Utils.create_canister` signature with cycle amount as third argument suggests this might be the case.

- **(B)** `IC0Utils` does not internally manage cycles, in which case the installer creates canisters with **0 cycles**. After `install_code`, the new pool would freeze almost immediately and be unusable — though `_initCycles = 1.86T` meant for it is silently lost.

Without access to the `mo:commons/utils/IC0Utils` source, this audit cannot definitively determine which is the case.

- **Impact (if scenario B confirmed):**
  - Every pool created via an external installer is created with 0 cycles.
  - Pool would freeze almost immediately after `install_code`.
  - User attempts to use the pool would fail with "canister out of cycles."
  - Admin's deposited cycles for installer would not transfer to pools — they'd accumulate in the installer.
  - For each pool creation: `1.86T + 0.5T = 2.36T` cycles "consumed" from caller balance but never landing in the new canister.
- **Verification step:**

  Open `mo:commons/utils/IC0Utils.mo` and inspect `create_canister` and `deposit_cycles` signatures. If either body contains `ExperimentalCycles.add(...)` or `system "ic0" "call_cycles_add"`, this finding is downgraded to Informational/redundant. If neither does, this is a confirmed High.
- **Recommendation:**

  Regardless of which scenario is correct, the inconsistency between factory and installer code paths is a code-smell. Standardize:

```
Cycles.add<s>(_initCycles);
let result = await IC0Utils.create_canister(null, null, _initCycles);
Cycles.add<s>(_initTopUpCycles);
await IC0Utils.deposit_cycles(result.canister_id, _initTopUpCycles);
```

  Add a unit test that creates a pool via the external installer path and asserts the new canister's cycle balance is approximately `_initCycles + _initTopUpCycles`.

- **Status:** New finding (severity contingent on `IC0Utils` source verification).

Medium (8)

NEW7-02 — `DeletedSwapPool.retryFailedRefunds` lacks `_isRefunding` mutex check

- **Severity:** Medium (controller-only attack surface)
- **Location:** `DeletedSwapPool.mo:142-158`
- **Description:**

  `refundAll` correctly checks `if (_isRefunding) { return "error=Refund already in progress..."; }` at line 49–51 to prevent re-entry while the timer-driven `_processRefund` is still running. The peer function `retryFailedRefunds` does not have this check:

```
public shared ({ caller }) func retryFailedRefunds() : async Text {
    if (not Principal.equal(caller, _controller)) { return "error=..."; };
    if (_failedRefunds.size() == 0) { return "No failed refunds to retry."; };
    // No _isRefunding check!
    _tokenHolderState := { token0; token1; balances = _failedRefunds };  // ov
    _failedRefunds := [];
    _refundIndex := 0;                                                    // r
    _isRefunding := true;                                                 // a
    ...
    ignore Timer.setTimer<s>(#nanoseconds(0), _processRefund);            // s
    return "Retry started. ...";
};
```

  If the controller calls this while `_processRefund` is still running (which it can be — `_processRefund` chains to itself with 500ms timers between users), the function:

  1. Overwrites `_tokenHolderState.balances` with `_failedRefunds`.
  2. Resets `_refundIndex` to 0.
  3. Schedules another `_processRefund` instance.

  The previously-running `_processRefund` instance, on its next iteration, reads `balances[_refundIndex]` against the new (overwritten) array starting from index 0. Two parallel processors may now fire `tokenAct.transfer` calls for the same user.

- **Impact:**

- **Out-of-order processing**: balances from the original pool that were not yet processed are silently lost (the original `_tokenHolderState.balances` is gone).
- **Possible double transfer to a user** if both processors process the same `_failedRefunds` entry simultaneously (token canister deduplication via `created_at_time = null` cannot help here; only memo-based dedup helps if memo is set, which it isn't).
- **State inconsistency observable via `getRefundStatus()` / `getRemainingBalances()`.**

Controller-only, so this is operator-error territory rather than an attack vector.

- **Recommendation:**

Add the `_isRefunding` mutex check at the top of `retryFailedRefunds`, mirroring `refundAll`:

```
public shared ({ caller }) func retryFailedRefunds() : async Text {
    if (not Principal.equal(caller, _controller)) { return "error=..."; };
    if (_isRefunding) { return "error=Refund still in progress. Wait for compl
    if (_failedRefunds.size() == 0) { return "No failed refunds to retry."; };
    ...
};
```

- **Status:** New finding (introduced by v6.2 NEW-25 fix).

NEW7-03 — `DeletedSwapPool.postupgrade` resumes refund without re-fetching token fee cache

- **Severity:** Medium (operational; no fund loss)
- **Location:** `DeletedSwapPool.mo:38-39, 250-257`
- **Description:**

`_token0Fee` and `_token1Fee` are declared as **non-stable** `private var ... = 0`:

```
private var _token0Fee : Nat = 0;
private var _token1Fee : Nat = 0;
```

`refundAll` correctly fetches these via `await _token0Act.fee()` / `await _token1Act.fee()` at line 56–57 before scheduling `_processRefund`.

`postupgrade` (added by the v6.2 NEW-25 fix to resume an interrupted refund) does not:

```
system func postupgrade() {
    _refundLogBuffer := Buffer.fromArray(_refundLog);
    _refundLog := [];
    // Resume refunding if interrupted by upgrade
    if (_isRefunding) {
        ignore Timer.setTimer<s>(#nanoseconds(0), _processRefund);
    };
};
```

After upgrade, `_token0Fee` and `_token1Fee` are reset to `0`. The resumed `_processRefund` operates with `fee = 0`, which causes:

- The condition `if (ab.balance0 > _token0Fee)` becomes "if balance0 > 0" — almost always true.
- `let amount0 = ab.balance0 - _token0Fee = ab.balance0` — full balance is sent (no fee subtracted).
- `tokenAct.transfer(... fee = ?0 ...)` — token canister rejects with `#Err(BadFee { expected_fee = N })`.
- All transfers fail, every user lands in `_failedRefunds`.

The resumed pass therefore terminates with all balances re-queued in `_failedRefunds`, having made zero successful transfers. No funds are lost (because the failed transfers never happened), but the refund pipeline is stuck until the controller manually triggers a fresh `refundAll` (which won't work because there are no balances left in `_tokenHolderState.balances`) or `retryFailedRefunds` (which has its own bug per NEW7-02).

- **Impact:**
  - **Refund pipeline bricked after upgrade interruption** until manual operator intervention.
  - User funds remain custodied by `DeletedSwapPool` but inaccessible until controller diagnoses and runs the right sequence.
  - Combined with NEW7-02, the controller's recovery is fragile.
- **Recommendation:**

  Refresh fee cache in `postupgrade` before scheduling resumption:

```
system func postupgrade() {
    _refundLogBuffer := Buffer.fromArray(_refundLog);
    _refundLog := [];
    if (_isRefunding) {
        ignore Timer.setTimer<s>(#nanoseconds(0), func() : async () {
            try { _token0Fee := await _token0Act.fee(); } catch (_) {};
            try { _token1Fee := await _token1Act.fee(); } catch (_) {};
            await _processRefund();
        });
    };
};
```

Alternatively, mark `_token0Fee` and `_token1Fee` as `stable` so they survive upgrade. (If the token's actual fee has changed between sessions, the stale value would also fail — but at least with a meaningful error, not BadFee 0.)

- **Status:** New finding (introduced by v6.2 NEW-25 fix).

NEW7-04 — `_autoDecrease` max-retry exit path skips `_pendingGeneration` increment

- **Severity:** Medium (correctness / consistency with v6.2 NEW-02 fix description)
- **Location:** `SwapPool.mo:258-275`
- **Description:**

  The v6.2 NEW-02 fix description states: *" `_pendingGeneration` … Incremented when **setting pending and when clearing pending**."*

  The fix correctly increments at:

  - Line 288: when a new pending is *set* (`_pendingGeneration += 1`).
  - Line 341: when pending is cleared after success.

  It does **not** increment at line 264 — the third path that clears pending, when max-retry is reached:

```
case (?pending) {
    _pendingRetryCount += 1;
    if (_pendingRetryCount >= _MAX_LIMIT_ORDER_RETRIES) {
        _failedLimitOrderBuffer.add(pending);
        _pendingExecution := null;     // pending is cleared
        _pendingRetryCount := 0;
```

```
            // _pendingGeneration += 1;    <-- MISSING
        } else {
            ignore Timer.setTimer<s>(#nanoseconds(0), _executeAutoDecrease);
            let gen = _pendingGeneration;
            ignore Timer.setTimer<s>(#seconds(10), func() : async () {
                if (gen == _pendingGeneration) { await _autoDecrease(); };
            });
            return;
        };
    };
```

Consequence: after the max-retry exit, an old watchdog (whose closure captured the now-stale generation `gen = N`) may still fire 10 seconds later. It checks `gen == _pendingGeneration` — which is still `N` because we didn't bump — so the watchdog proceeds and calls `_autoDecrease()`.

In practice the spurious watchdog enters the `case null { }` branch (because `_pendingExecution := null`) and falls through to scan the stack, which is harmless. But:

1. It violates the v6.2 fix's stated invariant.
2. If a future change to `_autoDecrease` makes the `case null` branch non-trivial, it becomes a bug.
3. It produces a subtle log/observability artifact — an unexpected `_autoDecrease` call appearing without a corresponding new pending.

- **Impact:** Minor functional impact today. Defensive consistency with v6.2 fix is broken.
- **Recommendation:**

```
if (_pendingRetryCount >= _MAX_LIMIT_ORDER_RETRIES) {
    _failedLimitOrderBuffer.add(pending);
    _pendingExecution := null;
    _pendingRetryCount := 0;
    _pendingGeneration += 1;       // <-- ADD
    // Fall through to process remaining stack items
};
```

Apply the same review to `setLimitOrderAvailable(false)` path at SwapPool.mo:194 — see **NEW7-13**.

- **Status:** New finding (v6.2 NEW-02 fix incomplete in third clear-path).

NEW7-10 — `PasscodeManager.depositFrom` lacks `_addLog` for ambiguous catch (NEW-19 partial)

- **Severity:** Medium
- **Location:** `PasscodeManager.mo:147-168`
- **Description:**

  V6.2 NEW-19 added `_addLog` to `deposit`'s catch (line 142) so that ambiguous transfer failures are visible to the operator. The peer function `depositFrom` was not updated:

```
public shared ({ caller }) func depositFrom(args : DepositArgs) : async Result
    ...
    try {
        switch (await TOKEN.transferFrom({...})) {
            case (#Ok(_)) { _walletDeposit(caller, args.amount); return #ok(ar
            case (#Err(msg)) { return #err(...); };
        };
    } catch (e) {
        let msg : Text = debug_show (Error.message(e));
        return #err(#InternalError(msg));        // <-- no _addLog!
    };
};
```

  When `transferFrom` traps in an ambiguous way (e.g., token canister upgrading mid-call, replica error, network instability), the user's wallet may have been debited but the pool's internal `_walletDeposit` was not called. With no log, the operator cannot detect this case and reconcile via the governance `transfer` function.

- **Impact:** Same as the V6 NEW-09 / NEW-19 cluster: silent ambiguous loss of user balance pending reconciliation. Detection currently impossible from logs.
- **Recommendation:**

```
} catch (e) {
    let errMsg = Error.message(e);
    _addLog(caller, "depositFrom transferFrom exception (ambiguous): " # errMs
    return #err(#InternalError(debug_show(errMsg)));
};
```

- **Status:** New finding (v6.2 NEW-19 fix incomplete; only `deposit` was patched).

NEW7-11 — `PositionIndex.removePoolIdWithoutCheck` is unauthenticated and bypasses position check

- **Severity:** Medium (data integrity, not funds)
- **Location:** `PositionIndex.mo:92-101`
- **Description:**

```
public shared (msg) func removePoolIdWithoutCheck(poolId : Text) : async Resul
    var user : Text = PrincipalUtils.toAddress(msg.caller);
    switch (_userPools.get(user)) {
        case (?poolArray) {
            _userPools.put(user, CollectionUtils.arrayRemove(poolArray, poolIc
        };
        case (_) {};
    };
    return #ok(true);
};
```

  This function:

  - Has no permission check.
  - Has no `Principal.isAnonymous` rejection.
  - Bypasses the `getUserPositionIdsByPrincipal` check that `removePoolId` performs.

  Anyone (including an anonymous principal) can dissociate themselves from any pool, even with active positions. The user's positions on the pool itself are not affected, but the indexing in `PositionIndex` is broken — frontends that rely on `getUserPools` to display a user's portfolio would no longer show the pool.

- **Impact:**
  - **User experience degradation**: a user accidentally calling this would lose their portfolio listing. The position itself is recoverable on the pool, but the user must remember which pool they have positions in.
  - **No fund loss**: positions remain on the pool itself.
  - **Spam vector**: anyone can spam-call this for the anonymous principal — limited harm because anonymous has no real positions.
- **Recommendation:**

  Either:

1. Restrict to admin: `_checkAdminPermission(msg.caller);`
2. Remove the public function entirely if it's only intended for migration use.
3. Or, if it's intentionally user-callable, add an explicit safety check (e.g., require an extra confirmation parameter): `removePoolIdWithoutCheck(poolId : Text, iUnderstandThisRemovesMyAssociation : Bool)`.

- **Status:** New finding.

NEW7-12 — `PositionIndex.addPoolId` allows any caller to create user→pool associations without owning a position

- **Severity:** Medium (data integrity)
- **Location:** `PositionIndex.mo:49-70`
- **Description:**

  `addPoolId` checks only that the `poolId` exists in `_poolIds` (the registered pool list). It does *not* verify that the caller actually has a position in that pool. Anyone — including an anonymous principal — can spam-add associations:

```
public shared (msg) func addPoolId(poolId : Text) : async Result.Result<Bool,
    var user : Text = PrincipalUtils.toAddress(msg.caller);
    if (ListUtils.arrayContains(_poolIds, poolId, Text.equal)) {
        // adds (user, poolId) to _userPools; no position-ownership verificati
        ...
        return #ok(true);
    };
    ...
};
```

  An attacker can iterate over all known pools and add bogus associations against the anonymous principal (or against rotating principals) to inflate the `_userPools` map.

- **Impact:**
  - HashMap growth → memory pressure → cycles cost.
  - On the order of *(num pools) × (num principals attacker generates)* entries possible.
  - No fund loss.
  - The auto-update timer `_autoUpdatePoolIds` runs every 300s but only refreshes

`_poolIds`, not `_userPools` — bogus associations persist.

- **Recommendation:**
  1. Reject `Principal.isAnonymous(msg.caller)`.
  2. Verify ownership via `await poolAct.getUserPositionIdsByPrincipal(msg.caller)` and reject if `.size() == 0`.
  3. Or, deprecate `addPoolId` in favor of `addPoolToUser` (which is pool-callable only) and let pools push associations.
- **Status:** New finding.

NEW7-13 — `setLimitOrderAvailable(false)` clears pending without `_pendingGeneration` bump

- **Severity:** Medium (consistency with NEW7-04; admin-only state hygiene)
- **Location:** `SwapPool.mo:183-198`
- **Description:**

  When admin disables limit orders, the function clears `_pendingExecution`:

```
public shared (msg) func setLimitOrderAvailable(available : Bool) : async () {
    ...
    if (not available and _isLimitOrderAvailable) {
        _limitOrderStack := List.nil<...>();
        _lowerLimitOrders := RBTree.RBTree<...>(_limitOrderKeyCompare);
        _upperLimitOrders := RBTree.RBTree<...>(_limitOrderKeyCompare);
        _pendingExecution := null;       // <-- cleared
        _pendingRetryCount := 0;
        // _pendingGeneration += 1;      <-- MISSING (same issue as NEW7-04)
    };
    _isLimitOrderAvailable := available;
};
```

  Same root cause as NEW7-04: a watchdog timer scheduled before `setLimitOrderAvailable(false)` will not be invalidated by generation mismatch and may fire after up to 10 seconds.

- **Impact:**

  When the watchdog fires, it calls `_autoDecrease()` which sees `_pendingExecution =`

`null` and falls through to scan the empty stack → harmless. But if limit orders are quickly re-enabled (`setLimitOrderAvailable(true)`) and a new pending is set with `_pendingGeneration` un-incremented, an old watchdog could falsely consider itself "current" and double-trigger.

With current values of `_MAX_LIMIT_ORDER_RETRIES = 3` , the practical impact is bounded: a real legitimate watchdog would still be needed to actually advance state.

- **Recommendation:**

  Add `_pendingGeneration += 1;` in the disable branch, alongside the v6.2 NEW-02 invariant.

- **Status:** New finding.

NEW7-14 — `SwapPool.postupgrade` resumes pending limit-order execution but `_pendingRetryCount` carries over

- **Severity:** Medium (operational correctness)
- **Location:** `SwapPool.mo:206-207, 3262-3285`
- **Description:**

  The pool's stable variables include:

```
private stable var _pendingExecution : ?(LimitOrderType, LimitOrderKey, LimitC
private stable var _pendingRetryCount : Nat = 0;
private var _pendingGeneration : Nat = 0;   // non-stable
```

  At `postupgrade` :

```
if (Option.isSome(_pendingExecution) or not List.isNil(_limitOrderStack)) {
    ignore Timer.setTimer<s>(#nanoseconds(0), _autoDecrease);
};
```

  When `_autoDecrease` runs after upgrade, it finds `_pendingExecution = ?pending` (preserved across upgrade) and `_pendingRetryCount = N` (also preserved). It increments to `N+1` and may immediately exceed `_MAX_LIMIT_ORDER_RETRIES = 3` , marking a legitimate limit order as failed.

  This is unfair: an upgrade interruption is a legitimate operational event, not an indication of a misbehaving order. The order should be retried fresh, not penalized.

- **Impact:**

  Limit orders that happen to be in `_pendingExecution` at upgrade time may be moved to `_failedLimitOrderBuffer` after only 1 (post-upgrade) attempt instead of 3 — possibly 0 attempts if `_pendingRetryCount` was already 2 before upgrade.

  User can recover via `removeLimitOrder` and re-add, but the user experience is poor.

- **Recommendation:**

  In `postupgrade`, reset retry counters before re-scheduling:

```
if (Option.isSome(_pendingExecution) or not List.isNil(_limitOrderStack)) {
    _pendingRetryCount := 0;          // give it a fresh start
    ignore Timer.setTimer<s>(#nanoseconds(0), _autoDecrease);
};
```

- **Status:** New finding.

Low (9)

NEW7-05 — `WasmManager.uploadChunk` lacks accumulated-size check

- **Severity:** Low (admin-only attack surface)
- **Location:** `components/WasmManager.mo:25-34`
- **Description:** `uploadChunk` only checks `assert(chunk.size() > 0)`. Cumulative size is checked in `combineChunks` (`assert(totalLength <= MAX_WASM_SIZE)`). An admin can upload arbitrarily many chunks before calling `combineChunks`, bloating heap memory.
- **Impact:** Heap pressure / cycle waste; admin-only.
- **Recommendation:** Track cumulative size per chunk and assert below MAX before storing each chunk.

NEW7-06 — `SwapFactory._parseVersion` uses `Prim.trap` instead of `Result`

- **Severity:** Low (UX)
- **Location:** `SwapFactory.mo:424-433`
- **Description:** Inconsistent with project's general `Result`-style error handling. Admin sees a trap rather than a structured error if version is malformed.

- **Recommendation:** Convert to `Result.Result<[Nat], Text>`; have `setNextPoolVersion` propagate via `throw Error.reject(...)`.

NEW7-15 — `_decreaseLiquidity` uses direct `Prim.trap` instead of `_rollback`

- **Severity:** Low (consistency)
- **Location:** `SwapPool.mo:691, 695`
- **Description:** Other failure paths use `_rollback("...")` (which is `Prim.trap`), but these two branches use `Prim.trap("...")` directly. Functionally equivalent but inconsistent style.
- **Recommendation:** Replace with `_rollback(...)`.

NEW7-16 — `_swap` private function has unreachable `#err` branch in callers

- **Severity:** Low (dead code / future refactoring trap)
- **Location:** `SwapPool.mo:1109–1132`, callers at 1784, 1865
- **Description:** `_swap` body is `try { ... } catch (e) { _rollback(...); }` — it always either traps or returns `#ok`. The `case (#err(e)) { ... }` branches in `depositFromAndSwap` and `depositAndSwap` are unreachable.
- **Impact:** None functionally. Future refactor that makes `_swap` return `#err` (without trapping) would not benefit from the existing handlers' state-fix logic — they were not exercised in any test.
- **Recommendation:** Either (a) delete the `#err` branches and document that `_swap` always traps or returns `#ok`, or (b) refactor `_swap` to return `#err` instead of trap, exercising the existing handlers.

NEW7-17 — `SwapDataBackup.isBackupDone` query is unauthenticated

- **Severity:** Low (information disclosure)
- **Location:** `SwapDataBackup.mo:177–189`
- **Description:** Anyone can query backup completion status for any pool. V6.2 NEW-32 added permission to `getPoolBackup` and `getAllPoolBackups`, but `isBackupDone` was not protected.
- **Impact:** Reveals backup operational timing.

- **Recommendation:** Add `_hasPermission(caller)` check (will need to convert to `query ({ caller })`).

NEW7-18 — `_collect` clamps `tokensOwed` invariant violations to WARN log only

- **Severity:** Low (silent corruption fallback)
- **Location:** `SwapPool.mo:768-806`
- **Description:** When `userPositionInfo.tokensOwed0 > positionInfo.tokensOwed0` (which should never happen by invariant), the code logs a WARN and clamps to `positionInfo.tokensOwed0`. No further error escalation, no admin alert.
- **Impact:** A real invariant violation is masked. The clamp prevents trap (good), but root cause goes uninvestigated.
- **Recommendation:** Increase visibility (e.g., persistent error counter, query method to expose).

NEW7-19 — `setUpgradePoolList` silently skips unknown pool IDs

- **Severity:** Low (UX / observability)
- **Location:** `SwapFactory.mo:507-543`
- **Description:** If admin passes a `poolId` not present in `_poolDataService.getPools()`, the inner `for` loop simply doesn't add the upgrade task. No error returned, no log emitted.
- **Recommendation:** Return a `Result.Result<{ added : [Principal]; skipped : [Principal] }, ...>` so the admin can confirm intent.

NEW7-20 — `_burnICS` uses `fee = null`, relying on token-specific burn semantics

- **Severity:** Low (tight coupling to ICS token implementation)
- **Location:** `SwapFeeReceiver.mo:347-374`
- **Description:** V6.2 NEW-31 retained `fee = null` "since burn has zero fee by design." This assumes the destination `governanceCid` triggers the ICS canister's special-case burn path. If the ICS canister is replaced with one that doesn't have this behavior, the transfer would fail with `BadFee`.
- **Recommendation:** Either pass an explicit fee (and trust governance to handle), or document the contract with a comment + assertion.

NEW7-21 — Inconsistent stable/non-stable mix for `_log` debug buffer in SwapPool

- **Severity:** Low (recovery hygiene)
- **Location:** `SwapPool.mo:65-77`
- **Description:**

```
private stable var _debugLogArray : [Text] = [];
private var _debugLog : Buffer.Buffer<Text> = Buffer.Buffer<Text>(0);
private stable var _debugLogWriteIndex : Nat = 0;
```

  `_debugLog` (Buffer) is non-stable, but its companion write-index is stable. After upgrade:

  - `preupgrade` writes Buffer.toArray to `_debugLogArray`, setting it as the persistence form.
  - `postupgrade` reads `_debugLogArray` back into `_debugLog`.
  - But `_debugLogWriteIndex` is independently stable, possibly out of sync with the new `_debugLog.size()`.

  The circular-buffer arithmetic in `getDebugLog` can produce incorrect chronological ordering if `_debugLog.size()` and `_debugLogWriteIndex` mismatch.

- **Recommendation:** Either make `_debugLog` indirectly stable via array, or make `_debugLogWriteIndex` non-stable and recompute from `_debugLogArray.size()` in postupgrade.

Informational (3)

NEW7-22 — Multiple `getCycleInfo` functions are unauthenticated

- **Severity:** Informational
- **Location:** `PositionIndex.mo:212`, `SwapDataBackup.mo:210`, `TrustedCanisterManager.mo:49`, `PasscodeManager.mo:298`, others
- **Description:** Cycle balance is publicly queryable on most canisters. Reveals operational metadata (canister activity, age estimate from cycle burn rate).
- **Recommendation:** Either gate behind admin or accept as low-sensitivity.

NEW7-23 — `_syncPools` clears `_tokenClaimLog` and `_tokenSwapLog` on every sync

- **Severity:** Informational (audit trail loss)
- **Location:** `SwapFeeReceiver.mo:651-655`
- **Description:** On each `_syncPools` cycle (every 24h+), the in-memory log buffers are reset.
- **Impact:** Operator cannot retroactively investigate fee flows from prior cycles via on-chain query.
- **Recommendation:** Move to a circular buffer (capped persistent log) so the most recent N cycles' data are retained.

NEW7-24 — `_executeSwap` ignores `_tokenHolderService.swap` return; pre-check is the only safety

- **Severity:** Informational (defense-in-depth)
- **Location:** `SwapPool.mo:1369, 1375`
- **Description:** Both `ignore _tokenHolderService.swap(caller, ..., ...)` calls discard the boolean result. The current safety relies on `_computeSwap(args, caller, true)` upstream `_checkAmount` (line 1178). If a future refactor allows `_executeSwap` to be reached without a balance pre-check, the swap silently moves pool reserves without debiting the user — breaking AMM invariants for other users.
- **Recommendation:** Replace `ignore` with `assert` or check-and-trap, e.g.:

```
if (not _tokenHolderService.swap(caller, _token0, ..., _token1, swapAmount)) {
    Prim.trap("Internal error: tokenHolder.swap returned false despite pre-che
};
```

  This adds a defense-in-depth layer with negligible cost.

ACCEPTED v6.2 ITEMS — Re-affirmed

The 5 items v6.2 marked ACCEPTED were re-evaluated and remain ACCEPTED with no change in risk posture:

ID	Item	Re-affirm Notes

NEW-07	WasmManager state non-stable	Within-session uploader isolation is sufficient.
NEW-09	PasscodeManager catch branches don't refund	Governance `transfer` provides reconciliation; **note newly identified gap NEW7-10**.
NEW-12	PositionIndex.updatePoolIds unauthenticated	Read-only sync. **Note adjacent NEW7-11/NEW7-12 issues with sibling functions**.
NEW-14	FullMath.mulDiv lacks denominator=0/overflow checks	All call sites guard upstream.
NEW-27	DeletedSwapPool single hardcoded controller	IC platform controller provides ultimate recovery.

ARCHITECTURAL OBSERVATIONS

These are not findings, but design observations that may be useful for the maintainers.

A1 — Heavy reliance on trap-based rollback

Many critical sections rely on `Prim.trap` (via `_rollback`) for state rollback, exploiting Motoko's message atomicity. Examples: `_executeSwap` partial state writes, `_updatePosition` partial tick updates, `_addLiquidity` failure path.

This is sound *under Motoko's current single-message atomicity guarantee*. It is fragile to:

- Future Motoko semantic changes.
- Refactors that introduce an `await` between the partial write and the trap.
- Errors in junior contributors who may extract a "helper" function and accidentally cross a message boundary.

Recommendation: Add a top-of-file comment explicitly documenting the trap-rollback contract for these functions, and consider an `assert` at entry that no `await` has been made since the last commit point.

A2 — Internal vs. external balance accounting

The pool maintains an internal `_tokenHolderService` ledger separate from the actual token canister balances. The two are kept in sync via a strict invariant: every credit to the internal ledger has a corresponding inbound transfer; every debit corresponds to an outbound

transfer.

This is the standard DEX architecture and is correct. However, the system has many places where this invariant could be silently broken:

- Ambiguous catch on inbound transfers (see NEW7-10, NEW-09 ACCEPTED).
- The `_refund` function debits the internal ledger before scheduling the external transfer, decoupling them across messages.
- `deleteFailedTransaction` 's admin path *assumes* that the failed deposit's transferFrom actually succeeded, but cannot verify this without an external token-canister query.

A protocol-level audit tool — a periodic job that queries token canister balance and compares to `Σ(internal balances) + Σ(protocol fees)` — would provide a strong invariant check. Currently no such reconciliation exists on-chain.

A3 — V3 state machine complexity

The transaction state machines (`Deposit` , `OneStepSwap` , `Refund` , etc.) carry significant complexity. The `OneStepSwap` flow in particular has 7 distinct status values plus three sub-status fields. Maintenance burden is high; a formal verification pass would be valuable.

A4 — Observability gaps

Multiple silent failure paths log via `_log` (debug ring buffer) but do not surface to administrators:

- `refundFailed` (timer-driven; admin not notified).
- `withdrawFailed` (timer-driven; admin not notified).
- `_pendingRetryCount` exhaustion → buffered to `_failedLimitOrderBuffer` , exposed via `getFailedLimitOrders` , but no alerting.
- `_clearExpiredFailedTransactionJob` warns about non-failed expired txs, but no alert.

A canister-level "health check" query that returns counts of stuck states (failed refunds, failed limit orders, expired txs) would let operators set up alerting.

FIX PRIORITY

P0 — Must Fix (User-visible / control-plane integrity)

ID	Title
NEW7-01	`await _refund` cannot detect transfer failure
NEW7-09	SwapPoolInstaller missing `Cycles.add` (pending `IC0Utils` verification)

P1 — Next Release (Operational risk / value leak)

ID	Title
NEW7-07	`_commonSwap` / `_ICRC1SwapToICP` no slippage protection
NEW7-02	`retryFailedRefunds` missing mutex
NEW7-03	`DeletedSwapPool.postupgrade` doesn't refresh fees
NEW7-08	`SwapDataBackup` non-atomic snapshot
NEW7-10	`PasscodeManager.depositFrom` missing ambiguous log
NEW7-14	`_pendingRetryCount` survives upgrade

P2 — Code Quality / Consistency

ID	Title
NEW7-04	`_autoDecrease` max-retry generation skip
NEW7-11	`removePoolIdWithoutCheck` unauthenticated
NEW7-12	`addPoolId` no ownership verification
NEW7-13	`setLimitOrderAvailable(false)` generation skip
NEW7-05	`uploadChunk` cumulative size
NEW7-15	`_decreaseLiquidity` direct trap
NEW7-18	`_collect` invariant clamp visibility
NEW7-21	`_log` stable/non-stable mismatch

P3 — Documentation / Minor

ID	Title
NEW7-06	`_parseVersion` uses trap
NEW7-16	`_swap` dead `#err` branches
NEW7-17	`isBackupDone` unauthenticated
NEW7-19	`setUpgradePoolList` silent skip
NEW7-20	`_burnICS fee = null` documentation
NEW7-22	`getCycleInfo` unauthenticated
NEW7-23	`_syncPools` log clearing
NEW7-24	`_executeSwap` ignored swap result

AUDIT SCOPE STATEMENT

Audited files (full or substantial):

- `src/SwapPool.mo` (3,323 LOC) — full pass
- `src/SwapFactory.mo` (1,171 LOC) — full pass
- `src/SwapFeeReceiver.mo` (974 LOC) — full pass
- `src/PasscodeManager.mo` (354 LOC) — full
- `src/PositionIndex.mo` (296 LOC) — full
- `src/SwapDataBackup.mo` (319 LOC) — full
- `src/SwapPoolInstaller.mo` (137 LOC) — full
- `src/DeletedSwapPool.mo` (258 LOC) — full
- `src/TrustedCanisterManager.mo` (64 LOC) — full
- `src/components/transaction/lib.mo` (852 LOC) — substantial
- `src/components/transaction/{Deposit,OneStepSwap,Refund,...}.mo` — full
- `src/components/{ICRC21,Job,TokenHolder,WasmManager,PositionTick}.mo` — full or substantial

- `src/utils/{AccountUtils,PoolUtils}.mo` — full
- `src/libraries/FullMath.mo` — full

Files NOT independently audited this round (relying on prior v6.2 coverage):

- `src/Types.mo` — data definitions only
- `src/libraries/{TickMath, SwapMath, SqrtPriceMath, LiquidityMath, LiquidityAmounts, Tick, TickBitmap, BlockTimestamp, FixedPoint128}.mo` — Uniswap V3 math primitives
- `src/components/{TokenAmount, SwapRecord, UpgradeTask, PoolData}.mo` — supporting components
- `src/SwapFactoryValidator.mo` — non-runtime validator

External dependencies treated as trusted:

- `mo:base/*` — Motoko base library
- `mo:commons/*` — including `IC0Utils`, `SafeUint`, `SafeInt`. **Note: NEW7-09 is contingent on `IC0Utils.create_canister` / `deposit_cycles` semantics, which were not directly inspected.**
- `mo:token-adapter/*` — token standard adapter

SIGN-OFF

Audit version: **v7.0** Total findings: **24 new** + 1 v6.2 fix verification failure (NEW-10 escalated to NEW7-01) + 28 v6.2 fix verifications PASS + 5 v6.2 ACCEPTED re-affirmed.

This audit raises the v7.0 bar above v6.2 by reviewing not only the post-fix state of v6.2 items but also the surrounding code paths, upgrade flows, MEV exposure, atomicity guarantees, and ambiguous-catch handling at the highest standard of review.

The protocol's **core swap and AMM math** continues to look sound (no findings in this round in the math primitives, consistent with prior audits). The remaining issues are concentrated in:

1. The asynchronous-refund / ambiguous-catch surface (NEW7-01, NEW7-10).
2. The auxiliary canisters (`DeletedSwapPool`, `SwapDataBackup`, `SwapPoolInstaller`, `SwapFeeReceiver`).

3. Operational hygiene (generation counters, upgrade resumption, observability).

After P0 and P1 fixes, the protocol should be considered ready for re-evaluation and a v7.1 spot-check audit.

— *End of Report v7.0*